

1 Sobre a entrega do trabalho

São requisitos para atribuição de notas a este trabalho:

- Uso de um arquivo `makefile` para facilitar a compilação. Os professores executarão `make` e deverão obter o arquivo executável funcional com a sua solução. Este executável, cujo nome deverá ser `tp5`, deverá estar no subdiretório `tp5/`;
- Ao compilar, incluir pelo menos `-Wall -g`. Se não compilar, o trabalho vale zero. Haverá desconto por cada *warning*;
- Arquivo de entrega:
 - Deve estar no formato TAR comprimido (`.tgz`);
 - O arquivo `tgz` deve ser criado considerando-se que existe um diretório com o nome do trabalho. Por exemplo, este trabalho é o `tp5/`;
 - Então seu `tgz` deve ser criado assim:
 - * Estando no diretório `tp5/`, faça:
 - * `cd ..`
 - * `tar zcvf tp5.tgz tp5`
 - Desta maneira, quando os professores abrirem o arquivo `tgz` (com o comando `tar zxvf tp5.tgz`) terão garantidamente o diretório correto da entrega para poderem fazer a correção semi-automática.
 - O que colocar no arquivo `tgz`? Todos os arquivos que são necessários para a compilação, por isso se você usa arquivos além dos especificados, coloque-os também. Mas minimamente deve conter todos os arquivos `.c`, `.h` e o `makefile`;
 - Os professores testarão seus programas em uma máquina do departamento de informática (por exemplo, `cpu1`), por isso, antes de entregar seu trabalho faça um teste em máquinas do DInf para garantir que tudo funcione bem.

2 Objetivos

Este trabalho tem como objetivo implementar o tipo abstrato de dado “Fila de Prioridades”, usando uma lista encadeada simples.

Uma fila de prioridades é uma fila de itens onde cada item é associado a uma prioridade (ou chave); os itens da fila são mantidos ordenados segundo prioridades crescentes. Filas de prioridades são muito usadas quando há necessidade de manter dados ordenados

segundo algum critério. Por exemplo, simulações geralmente usam uma fila de prioridades chamada *Lista de Eventos Futuros* (LEF) para manter os próximos eventos que irão ocorrer na simulação, na ordem crescente do tempo associado a cada evento. Nesse caso, a chave/prioridade de cada evento é o seu instante de ocorrência.

A fila de prioridades a ser implementada neste projeto deve ser **genérica**, ou seja, capaz de armazenar itens de qualquer tipo. Para alcançar isso, a fila não deve armazenar os itens, mas somente ponteiros para eles. Assim, cada nó da lista encadeada deve conter um ponteiro genérico (`void *`), que aponta para o dado armazenado, e um inteiro usado para identificar o tipo daquele conteúdo.

Você deve baixar o arquivo `tp5.tgz` anexo a este enunciado e abri-lo para poder fazer o trabalho, pois irá precisar de todos os arquivos ali contidos:

- `fprio.h`: arquivo (read only) de *header* com todos os protótipos das funções para manipular filas de prioridades;
- `fprio.c`: um esqueleto de arquivo, a completar com as implementações das funções;
- `makefile`: sugestão de um makefile que você pode usar. É sua responsabilidade fazer as adaptações necessárias neste arquivo sugerido.
- `testa-fprio.c`: arquivo (read-only) de teste das filas.
- `testa-fprio.txt`: saída esperada do programa de teste.

Os arquivos `fprio.h` e `testa-fprio.*` não podem ser alterados. Na correção, os professores usarão os arquivos originais fornecidos.

- Use boas práticas de programação, como indentação, bons nomes para variáveis, comentários no código, bibliotecas, *defines*... Um trabalho que não tenha sido implementado com boas práticas vale zero.
- Quaisquer dúvidas com relação a este enunciado devem ser solucionadas via email para `prog1prof@inf.ufpr.br`, pois assim todos os professores receberão os questionamentos. Na dúvida, não tome decisões sobre a especificação, pergunte!
- Não envie mensagens pelo Moodle, os professores nem sempre estão logados na plataforma. As respostas serão mais rápidas se as mensagens vierem no e-mail acima.
- Dúvidas podem e devem ser resolvidas durante as aulas.

3 O que entregar

Entregue um único arquivo `tp5.tgz` que contenha por sua vez os seguintes arquivos:

- `fprio.h`: o mesmo arquivo fornecido, não o modifique;

- `fprio.c`: sua implementação do `fprio.h`;
- `testa-fprio.c`: o mesmo arquivo fornecido, não o modifique;
- `makefile`.

Atenção: Os arquivos `fprio.h` e `testa-fprio.c` não podem ser alterados. Na correção, os professores usarão os arquivos originais fornecidos.

4 Recomendações

- Faça o programa aos poucos, construindo e testando cada função em separado.
- O programa de teste `testa-fprio.c` aplica diversos testes às funções implementadas em `fprio.c`. Entenda bem seu conteúdo e as mensagens de erro que ele pode gerar, pois elas podem facilitar a identificação e localização de erros na sua implementação.
- Compare a saída de seu programa com a saída esperada que está no arquivo `testa-fprio.txt`. Essa comparação pode ser facilmente realizada com o comando `diff`:

```
$ ./testa-fprio > minha-saida
$ diff testa-fprio.txt minha-saida
(eventuais diferenças entre os arquivos serão mostradas aqui)
```

Bom trabalho!